
Maven™

¿Qué es Maven?

- Maven es una herramienta Open Source desarrollada en Java por la asociación Apache Foundation, un organismo sin ánimo de lucro que publica recursos libres de derechos para asegurar el desarrollo y la innovación en las tecnologías de la información.
- El objetivo de Maven es **facilitar y estandarizar la organización y la construcción de proyectos de software**, principalmente para proyectos destinados a la máquina virtual de Java.
- Maven **simplifica las tareas de compilación y construcción** de aplicaciones de software, principalmente usado con lenguajes de programación que tienen como destino ser ejecutadas en la Java Virtual Machine (JVM). Además, incorpora una **gestión de dependencias** madura con acceso a los repositorios públicos de Maven con más de 16 Millones de librerías disponibles incluyendo todas las versiones de cada una de ellas.

¿Qué hace Maven?

- Identificación de paquetes
- Distribución de paquetes
- Gestión de dependencias
- Automatización de procesos de construcción complejos
- Independencia de los IDEs
- Ejecución de pruebas unitarias y de integración y reporte de resultados
- Construcción de proyectos con múltiples módulos

Instalación

- UBUNTU

  sudo apt install Maven

- WINDOWS
  - Descargar Maven (zip) - Pagina oficial proyecto Maven
  - Crear la variable MAVEN_HOME con la dirección de instalación

    C:\Users\tu_usuario>setx MAVEN_HOME <path_instalación> /M

    O desde variables de entorno
  - En variables de entorno, agregar al path %MAVEN_HOME%\bin

Instalación

- MAC OS
  - Descargar Maven (tar.gz) - Pagina oficial proyecto Maven
  - tar -zxvf apache-maven-version-bin.tar
  - Crear la variable MAVEN_HOME con la dirección de instalación

    Editar el fichero ~/.zshrc y agregar al final:

    Export MAVEN_HOME=“directorio_instalacion”

  - Agregarlo al path
    - En el mismo fichero ~/.zshrc agregar:

      export PATH="$PATH:$MAVEN_HOME/bin“
    - En caso de querer hacerlo para todos los usuarios, modificar /etc/paths y agregar al final

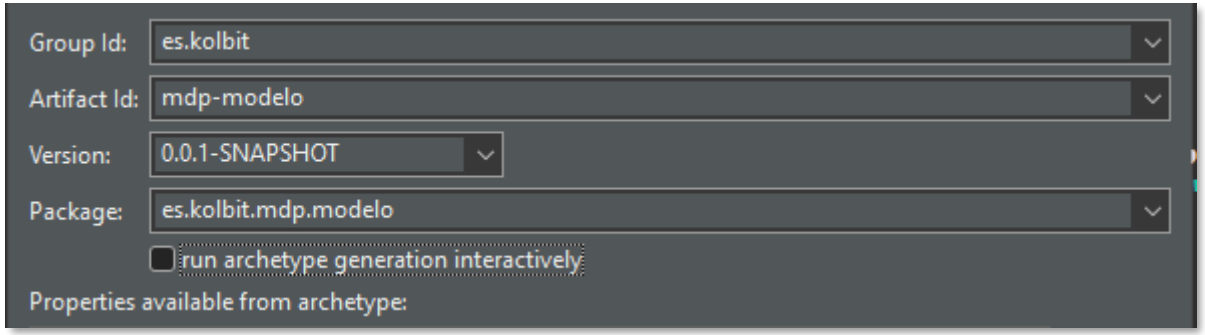
      $MAVEN_HOME/bin“

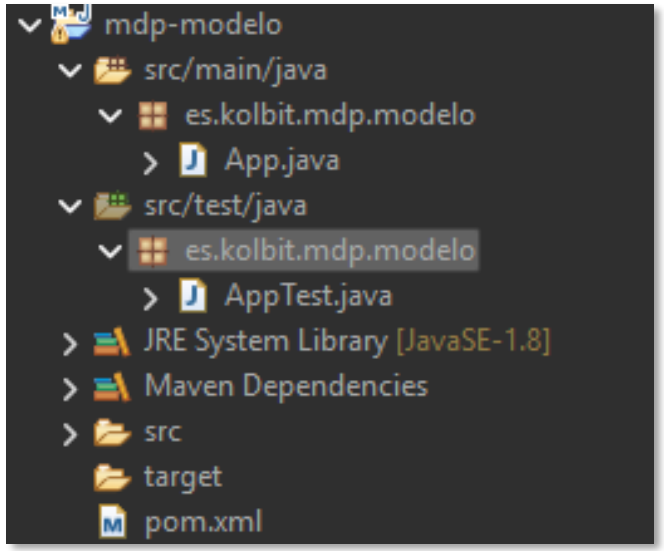
Coordenadas y dependencias

- Coordenadas Maven
  - groupId: normalmente identifica a la organización
  - artifactId: identifica el artefacto dentro de su grupo de desarrollo
  - version: es la versión del artefacto
- Artefactos en desarrollo
  - Sufijo –SNAPSHOT
  - Maven siempre consulta el repositorio remoto aunque esté en local si tiene este sufijo
- Se pueden especificar de forma declarativa las dependencias, haciendo referencia a sus coordenadas.

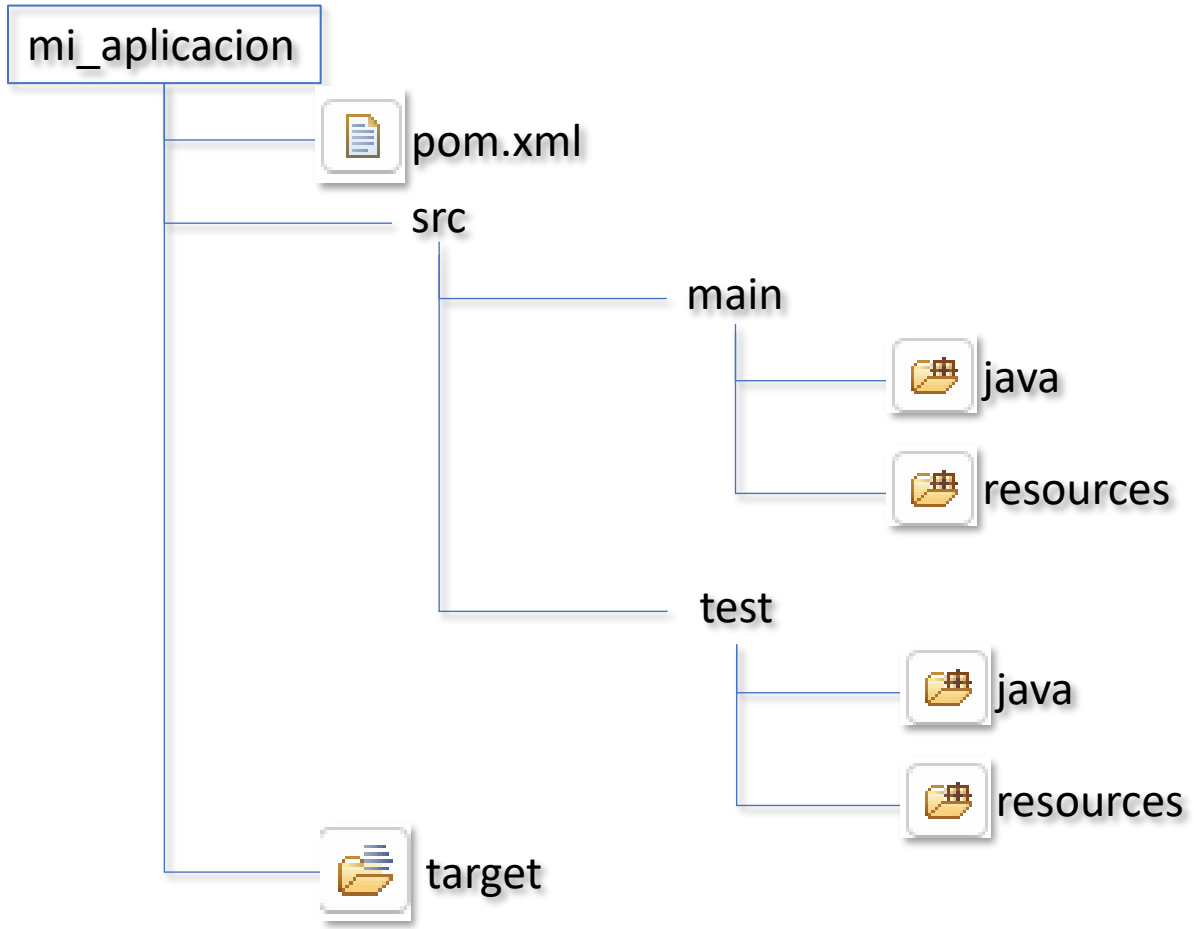
Creación proyecto Maven quickstart en Eclipse

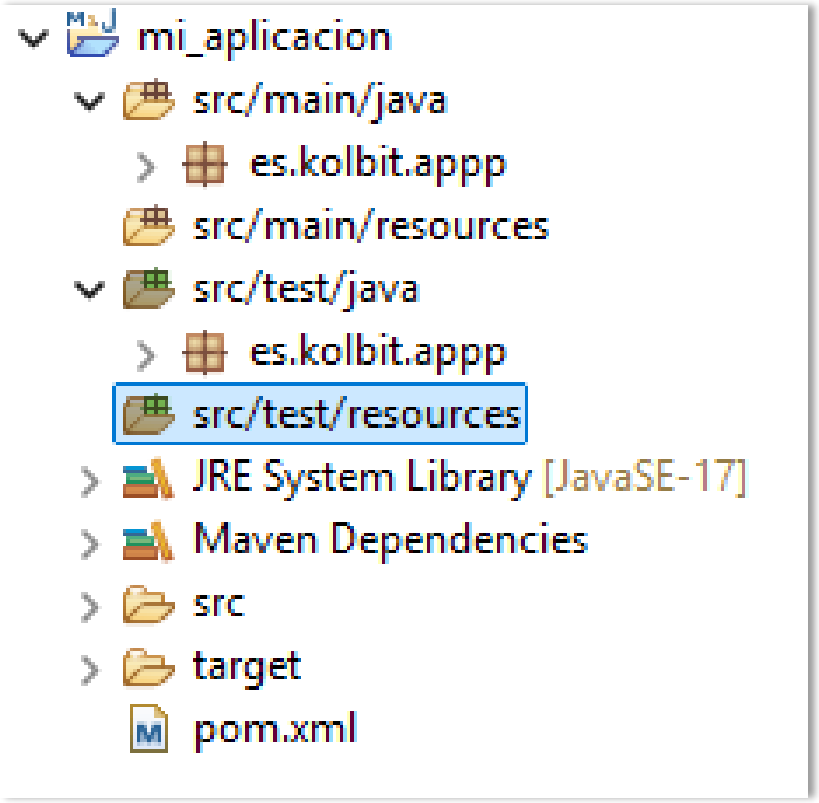
File > New > Maven Project
Create a simple project (debe estar desmarcado) > Next
Catalog: Maven Central
Filter: org.apache.maven
Seleccionar: maven-arcetype-quickstart Version 1.5 > Next





Proyecto Maven





Proyecto Maven

- **pom.xml**: fichero de especificación y configuración, ubicado en la raíz del proyecto
- **src/main/java**: directorio en donde se ubicarán los ficheros Java del proyecto (fuentes).
- **src/main/resources**: permite añadir recursos que también se incorporarán dentro del binario resultante, pero que no deben ser compilados. Por ejemplo, los ficheros con extensión *.properties*, XML o imágenes.
- **src/test/java**: directorio para colocar las fuentes Java que únicamente se usarán en la ejecución de los test unitarios. El contenido de este directorio no será empaquetado en el binario final.
- **src/test/resources**: necesario si para la ejecución de los test se utilizan otros recursos.
- **target/classes**: en este directorio se almacenarán las clases Java resultantes de la compilación de las fuentes almacenadas en *src/main/java*.
- **target**/<proyecto.jar>: típicamente los proyectos generarán un binario con el contenido de *target/classes* más *src/main/resources* en un único archivo empaquetado con extensión *jar* o war.

Project Object Model (pom)

- El *project object model* es la unidad de trabajo fundamental de Maven para la especificación y configuración de un proyecto.
- Se trata de un fichero llamado normalmente pom.xml donde se especifica (en formato XML) las dependencias del proyecto, sus coordenadas y todas las instrucciones necesarias para la construcción del software.

pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns=http://maven.apache.org/POM/4.0.0 xmlns:xsi=http://www. …>
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.kolbit</groupId>
  <artifactId>mdp-modelo</artifactId>
  <version>1.0.0</version>
  <name>mdp-modelo</name>
  <description>A simple mdp-modelo.</description>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>5.11.4</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Dependencias <dependencies>

```xml
<dependencies>
    <!--https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-api --
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-api</artifactId>
        <version>5.11.4</version>
        <scope>test</scope>
    </dependency>
    <!--https://mvnrepository.com/artifact/org.springframework/spring-context -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>6.2.2</version>
        <scope>provided</scope>
    </dependency>
```

Ámbito de las dependencias <scope>

- **compile**: (valor por defecto) disponible en todos los classpath
- **runtime**: sólo disponible en ejecución y pruebas (no en compilación)
- **test**: disponible sólo en compilación de pruebas y pruebas de test
- **provided**: sólo en compilación y pruebas. En ejecución estará presente en el entorno de ejecución
- **system:** la dependencia está en el sistema de archivos. (no debe utilizarse)
- **import:** solo en <dependencyManagement> para importar dependencias de proyectos multi-módulo

Ámbito de las dependencias <scope>

Scope	Compilar	Ejecutar	Test	Propagación	Usos, ejemplos
compile	Sí	Sí	Sí	Sí	
runtime	—	Sí	Sí	Sí	Driver, sólo en ejecución
test	—	—	Sí	—	JUnit, Mokito
provided	Sí	—	Sí	—	Servidor de app
system	Sí	—	Sí	—	No utilizar
import	—	—	—	—	Importa desde otro pom

KolbIT

Ámbito de las dependencias <scope>

- Necesaria para compilar y ejecutar → **compile**
- La aporta el contenedor (Tomcat/Jakarta EE) → **provided**
- Solo para ejecutar (drivers, providers) → **runtime**
- Solo para pruebas → **test**
- Evitar → **system**
- BOM → **import** (solo en dependencyManagement)

Repositorios

- Siempre existe un repositorio local asociado al usuario. “.m2”
- Si Maven no encuentra una dependencia en el repositorio local, lo busca en repositorios remotos
- Los repositorios remotos se definen en el fichero “settings.xml”
- Si no se configura ningún repositorio remoto, Maven utiliza el “Maven Central”

Maven lifecycles

FASE	DESCRIPCIÓN
clean	Validar que el proyecto sea correcto y que toda la información necesaria esté disponible.
default	Inicializar el estado de compilación, por ejemplo, establecer propiedades o crear directorios.
site	Generar cualquier código fuente para incluirlo en la compilación.

Default Lifecycle

FASE	DESCRIPCIÓN
validate	Validar que el proyecto sea correcto y que toda la información necesaria esté disponible.
initialize	Inicializar el estado de compilación, por ejemplo, establecer propiedades o crear directorios.
generate-sources	Generar cualquier código fuente para incluirlo en la compilación.
process-sources	Procesar el código fuente, por ejemplo, para filtrar cualquier valor.
generate-resources	Generar recursos para incluirlos en el paquete.
process-resources	Copiar y procesar los recursos en el directorio de destino, listo para empaquetar.
compile	Compilar el código fuente del proyecto.
process-classes	Postprocesar los archivos generados a partir de la compilación, por ejemplo, para mejorar el código de bytes en las clases Java.
generate-test-sources	Generar cualquier código fuente de prueba para incluirlo en la compilación.
process-test-sources	Procesar el código fuente de prueba, por ejemplo, para filtrar cualquier valor.
generate-test-resources	Crear recursos para realizar pruebas.
process-test-resources	Copiar y procesar los recursos en el directorio de destino de la prueba.

Default Lifecycle

FASE	DESCRIPCIÓN
test-compile	Compilar el código fuente de prueba en el directorio de destino de la prueba
process-test-classes	Postprocesar los archivos generados a partir de la compilación de prueba, por ejemplo, para realizar mejoras de código de bytes en clases Java.
test	Ejecute pruebas utilizando un marco de pruebas unitarias adecuado. Estas pruebas no deberían requerir que el código esté empaquetado o implementado.
prepare-package	Realizar todas las operaciones necesarias para preparar un paquete antes de su embalaje real. Esto suele dar como resultado una versión procesada y sin embalaje del paquete.
package	Tomar el código compilado y empaquetarlo en su formato distribuible, como un JAR.
pre-integration-test	Realizar las acciones necesarias antes de ejecutar las pruebas de integración. Esto puede implicar cosas como configurar el entorno necesario.
integration-test	Procesar e implementar el paquete, si es necesario, en el entorno de ejecución pruebas de integración
post-integration-test	Realizar las acciones necesarias después de que se hayan ejecutado las pruebas de integración. Esto puede incluir la limpieza del entorno.
verify	Ejecute todas las comprobaciones necesarias para verificar que el paquete sea válido y cumpla con los criterios de calidad.
install	Instalar el paquete en el repositorio local, para usarlo como dependencia localmente.
deploy	Copia el paquete final al repositorio remoto para compartirlo con otros desarrolladores y proyectos.

Default Lifecycle

FASE	DESCRIPCIÓN
validate	Valida que el proyecto sea correcto y que toda la información necesaria esté disponible.
compile	Compila el código fuente del proyecto.
test	Ejecuta los tests unitarios utilizando un marco de pruebas unitarias adecuado.
package	Genera el artefacto. Toma el código compilado y lo empaqueta en su formato distribuible, como un JAR.
verify	Ejecuta todas las comprobaciones necesarias para verificar que el paquete sea válido y cumpla con los criterios de calidad.
install	Instala el paquete en el repositorio local, para usarlo como dependencia localmente.
deploy	Copia el paquete final en un repositorio remoto.

Clean Lifecycle

FASE	DESCRIPCIÓN
pre-clean	Ejecutar los procesos necesarios antes de la limpieza real del proyecto.
clean	Eliminar todos los archivos generados por la compilación anterior
post-clean	Ejecutar procesos necesarios para finalizar la limpieza del proyecto

Site Lifecycle

FASE	DESCRIPCIÓN
pre-site	Ejecutar los procesos necesarios antes de la generación del sitio del proyecto real
site	Generar la documentación del sitio del proyecto
post-site	Ejecutar los procesos necesarios para finalizar la generación del sitio y prepararse para la implementación del sitio.
site-deploy	Implementar la documentación del sitio generada en el servidor web especificado